Floating-Point (Smallest/Largest Numbers), Comparison with Fixed-Point

Sparsh Mittal

Floating-Point Example

- Represent –0.75
  - $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$
  - S = 1
  - Fraction = $1000\ldots00_2$
  - Exponent = –1 + Bias
    - Single: –1 + 127 = 126 = 01111110_2
    - Double: –1 + 1023 = 1022 = 01111111110_2
- Single: 10111111101000…00
- Double: 10111111111101000…00

Floating-Point Example

- What number is represented by the single-precision float

  1100000010100 0…00

  - S = 1
  - Fraction = $01000\ldots00_2$
  - Exponent = $10000001_2 = 129$
- $x = (-1)^1 \times (1 + .01_2) \times 2^{(129 - 127)}$

  $= (-1) \times 1.25 \times 2^2$

  $= -5.0$

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

Range of Normal Numbers (Single-Precision)

- Exponents 00000000 and 11111111 are reserved for special cases

Binary

		Exponent									Mantissa																							
Smallest	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Largest	0	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Decimal

- Smallest value
  - Excess Exponent: 00000001 $\Rightarrow$ actual exponent = 1 – 127 = –126
  - Fraction: 000…00 $\Rightarrow$ significand = 1.0
  - Smallest Value = $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Largest value
  - Excess exponent: 11111110. $\Rightarrow$ actual exponent = 254 – 127 = +127
  - Fraction: 111…11 $\Rightarrow$ significand= 1.11111111111111111111111 $\approx$ 2.0
  - Largest Value = $\pm\,(2 - 2^{-23}) \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Range of Normal Numbers (Double-Precision)

- Exponents 0000…00 and 1111…11 reserved
- Smallest value
  - Exponent: 00000000001
    $\Rightarrow$ actual exponent = 1 – 1023 = –1022
  - Fraction: 000…00 $\Rightarrow$ significand = 1.0
  - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Largest value
  - Exponent: 11111111110
    $\Rightarrow$ actual exponent = 2046 – 1023 = +1023
  - Fraction: 111…11 $\Rightarrow$ significand $\approx$ 2.0
  - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Range of Integer, FP, and fixed-point

- A signed 32-bit integer variable has a maximum value of $2^{31} - 1 = 2{,}147{,}483{,}647$,
- An IEEE 754 32-bit base-2 floating-point variable has a maximum value of $(2 - 2^{-23}) \times 2^{127} \approx 3.4028235 \times 10^{38}$.
- A floating-point variable can represent a wider range of numbers than a fixed point (or integer) variable of the same bit width at the cost of precision.

Why do we use bias?

Let us compute the range of single-precision number, without using bias.

Single-Precision Range **[if no bias is used]**

- Smallest value
  - Exponent: 00000001
    $\Rightarrow$ actual exponent = 1 – 0 = 1
  - Fraction: 000…00 $\Rightarrow$ significand = 1.0
  - $\pm 1.0 \times 2^{1} \approx \pm 2$
- Largest value
  - exponent: 11111110
    $\Rightarrow$ actual exponent = 254-0 = 254
  - Fraction: 111…11 $\Rightarrow$ significand $\approx$ 2.0
  - $\pm 2.0 \times 2^{+254} \approx \pm 2.9 \times 10^{+76}$

1. On not using bias, we can only represent large numbers, and not small numbers

Why do we use bias?

To represent small numbers, we can start using negative values. But that would be confusing

Use of a bias means, we store excess exponent, which is always positive.

Denormal Numbers

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-126}$$

Binary

		Exponent									Mantissa																							
Smallest	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
Largest	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Decimal

* Smallest denormal number :
  * $0.00..01 * 2^{-126} = (2^{-23}) * 2^{-126} = 2^{-149}$
* Largest denormal number :
  * $0.11...11 * 2^{-126} = (1 - 2^{-23}) * 2^{-126} = 2^{-126} - 2^{-149}$

➔ The range of denormal numbers is $[2^{-149}, 2^{-126} - 2^{-149}]$

➔ The largest denormal number ($2^{-126} - 2^{-149}$) is little smaller than the smallest normal number (2^{-126}).

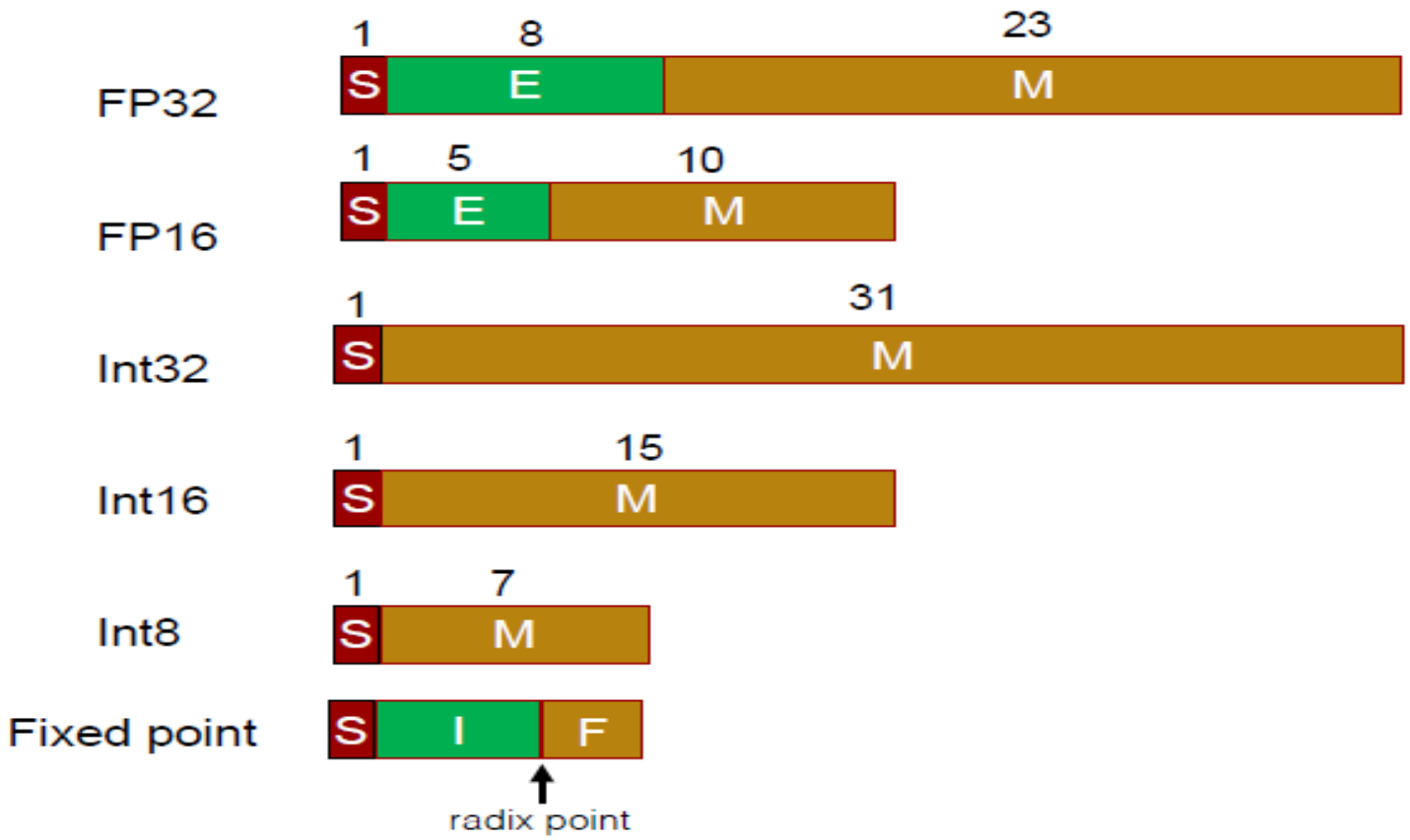
Solved example on Fixed Point Number

* In an application, all data values are positive (no sign bit required). The highest value we need to store is 36000. We want to use 20-bit storage and fixed-point number system. How many bits should we allocate for integer and fraction part, so we can have as high precision as possible, with no overflow.
* Solution: Since all data values are positive, no sign bit is needed. Now, 2^n-1>=36000
* n>= $\log_2(36001)$ ➔ n>=15.14
* Integer part: 16 bits, Fraction Part: 4 bits

Finding Percentage Error With Floating-Point Numbers

Quick Summary

$$(-1)^S \times (1.M) \times 2^E$$



	Range	Error
FP32	10^{-38} - 10^{38}	.000006%
FP16	6×10^{-5} - 6×10^{4}	.05%
Int32	0 – 2×10^{9}	½
Int16	0 – 6×10^{4}	½
Int8	0 – 127	½
Fixed point	-	-

Dally, High Performance Hardware for Machine Learning, NIPS'2015

Finding Percentage Error of FP32

- The smallest number: 1.17549435E-38
- The next smallest number: 1.17549449 E-38

- Find the difference between them: 1.4E-07 * E-38 . Call it Delta

- Find Delta *100/(SmallestNumber*2). This gives you percentage error, which is 5.95494E-06%, or 0.0000059549% or ~0.000006%.

How Pi Looks at Different Precision Levels

Double precision	0100000000001001001000011111101101010100010001000010110100011000
Single precision	01000000010010010000111111011011
Half precision	0100001001001000

Image courtesy: https://blogs.nvidia.com/blog/2019/11/15/whats-the-difference-between-single-double-multi-and-mixed-precision-computing/